



KIICHAIN BE - TAKE-HOME TEST

Estimated time: ~2 to 4 hours

Overview

The challenge is to create a simple HTTP service that accepts signed webhooks. The code must be written in Golang. This simulates an exchange, where balances are updated via webhooks sent from an external system.

The service should:

- Listen for incoming HTTP POST requests at the endpoint `/webhook`.
- Validate the signature of the incoming request using HMAC SHA256. The secret key for HMAC should be configurable.
- The signature will be sent in the `X-Signature` header of the request. The signature is an HMAC SHA256 hash of the request body.
- The webhook should prevent replay attacks with a nonce and timestamp mechanism.
- Each accepted request should be added to a ledger.

Constraints

These are the constraints for the implementation:

- The service must be implemented in Go.
- Use only standard libraries and publicly available packages.
- The service must handle concurrent requests safely.

Requirements:

1. Endpoint: `POST /webhook`

Accept a JSON body with:

```
{
  "user": "string",
  "asset": "string",
  "amount": "string (representing a decimal number)"
}
```

Headers:

- `X-Timestamp` : The UNIX timestamp of when the request was created.
- `X-Signature` : The HMAC SHA256 signature of the request body.
- `X-Nonce` : A unique nonce for each request.

Compute HMAC SHA256 as:

```
request_body = X-Timestamp + "\n" + X-Nonce + "\n" + <raw_request_body_bytes_as_string>  
HMAC_SHA256(secret_key, request_body)
```

Validation:

- X-Timestamp must be within 5 minutes of the current server time.
- X-Nonce must be unique for each request.
- Reject if the signature does not match.

If accepted:

- Apply the balance update to the user's ledger.
- Respond with HTTP 200 OK.

If rejected (invalid signature, replay attack, etc.):

- Respond with relevant HTTP status codes
- Endpoint: GET /balance/{user}

Return the user balance in JSON format:

```
{  
  "user": "string",  
  "balances": {  
    "asset1": "string (representing a decimal number)",  
    "asset2": "string (representing a decimal number)",  
    ...  
  }  
}
```

BONUS POINTS:

- Implement telemetry for incoming requests and errors.
- Structured logging with request IDs.
- Make the timestamp tolerance configurable.

Acceptance Criteria:

We will check:

- Correct execution of the requirements above.
- Code quality and organization (lint).
- Comments and documentation.
- Proper error handling and logging.
- Tests covering the main functionalities.
- Ledger handling of float precision for balances.

Submission

- Host your project on a **public GitHub repository**
 - Include a `README.md` with instructions to run it locally
 - Include a `SH` file that goes through the full flow of the project (Build the signature and send the request to the server)
-

If you have any questions or need clarification during the process, feel free to contact us.